

FCN - Indice documentale della libreria `fcn_lib_calc`

Documento di riferimento per le dichiarazioni pubbliche presenti nell'header `fcn_lib_calc.h`, con descrizioni tecniche sintetiche e note implementative derivate dalla libreria `fcn_lib_calc.cpp`.[\[file:16\]](#)[\[file:32\]](#)

Alias, costanti ed enumerazioni

```
using Vec = std::vector<double>
```

Alias per vettori reali in doppia precisione, usato come tipo base per segnali, nodi, colonne e autovalori.
[\[file:16\]](#)

```
using VecN = std::vector<int>
```

Alias per vettori interi, utile come contenitore di indici o contatori discreti.[\[file:16\]](#)

```
using Mat = std::vector<Vec>
```

Alias per matrici reali dense rappresentate come vettori di righe.[\[file:16\]](#)

```
using KM = matplot::keyword_manual_type
```

Alias di comodo per keyword manuali di Matplot++.[\[file:16\]](#)

```
using KA = matplot::keyword_automatic_type
```

Alias di comodo per keyword automatiche di Matplot++.[\[file:16\]](#)

```
const double PI = std::acos(-1.0);
```

Costante geometrica π calcolata tramite `acos(-1.0)`.[\[file:16\]](#)

```
enum class SortOrder { Asc, Desc };
```

Enumerazione per il riordino crescente o decrescente di autovalori, valori singolari e colonne associate.[\[file:16\]](#)
[\[file:32\]](#)

funzioni di gestione interfaccia utente

```
void cin_clear()
```

Pulisce lo stato di `std::cin` in caso di errore e scarta l'eventuale input residuo disponibile nel buffer.[\[file:16\]](#)
[\[file:32\]](#)

```
string color_bool(const bool val)
```

Restituisce una stringa ANSI di colore associata a un valore booleano, pensata per evidenziare esiti logici in output testuale.[\[file:16\]](#)[\[file:32\]](#)

```
string colordbl(const double val)
```

Restituisce una stringa ANSI di colore in funzione del segno e dell'ordine di grandezza del numero, usando soglie numeriche interne per distinguere valori positivi, negativi e quasi nulli.[file:16][file:32]

```
void color_rst()
```

Ripristina il colore standard del terminale emettendo il codice ANSI di reset.[file:16][file:32]

funzioni di conversione formato ed equivalenza unita' di misura

```
std::string itostr(const int nn)
```

Converte un intero in stringa tramite `std::to_string`. [file:16][file:32]

```
double deg2rad(const int alpha)
```

Converte un angolo espresso in gradi nel corrispondente valore in radianti usando la costante `PI`. [file:16][file:32]

```
string format_numstr(double v)
```

Formatta un numero reale in notazione scientifica con due cifre decimali significative nel mantissa output. [file:16][file:32]

funzioni di base per intervalli

```
double h_ticks(const double a_start, const double a_stop, const int a_points)
```

Calcola il passo uniforme di una griglia su intervallo chiuso come $(a_stop - a_start)/(a_points - 1)$. Termina l'esecuzione se l'intervallo non è valido. [file:16][file:32]

```
Vec nodi_bubblesort(const Vec x_uns, const int totnum)
```

Restituisce una copia ordinata in senso crescente del vettore di ingresso, mediante ordinamento elementare a confronti successivi. [file:16][file:32]

```
Vec nodi_equidistanti(const double amin, const double amax, const int NPoints)
```

Genera `NPoints` nodi equidistanti nell'intervallo `[amin, amax]` usando il passo calcolato da `h_ticks`. [file:16][file:32]

```
Vec nodi_random(const double amin, const double amax, const int NPoints)
```

Genera un vettore di nodi pseudo-casuali scalando campioni prodotti da `std::mt19937` sull'intervallo richiesto. [file:16][file:32]

funzioni matematiche ad uso callback

```
double f_x(double t)
```

Callback identità, restituisce il valore t .[\[file:16\]](#)[\[file:32\]](#)

```
double f_x1(double t)
```

Callback identità traslata, restituisce $t - 1$. Usata come coefficiente $at(t)$ nei sistemi di Cauchy con termine lineare non omogeneo.[\[file:16\]](#)[\[file:32\]](#)

```
double at_zero(double t)
```

Callback costante nulla per il coefficiente di $z(t)$ nel sistema di Cauchy: restituisce 0.0 .[\[file:16\]](#)[\[file:32\]](#)

```
double at_one(double t)
```

Callback costante unitaria per il coefficiente di $z(t)$ nel sistema di Cauchy: restituisce 1.0 . Usata come default in `matrix_build_cauchy`.[\[file:16\]](#)[\[file:32\]](#)

```
double ft_zero(double t)
```

Callback sorgente nulla: restituisce 0.0 . Usata come termine forzante di default in `matrix_build_cauchy`.[\[file:16\]](#)[\[file:32\]](#)

```
double f_sin(double t)
```

Callback seno, restituisce $\sin(t)$.[\[file:16\]](#)[\[file:32\]](#)

```
double f_cos(double t)
```

Callback coseno, restituisce $\cos(t)$.[\[file:16\]](#)[\[file:32\]](#)

```
double f_atan(double t)
```

Callback arcotangente, restituisce $\text{atan}(t)$.[\[file:16\]](#)[\[file:32\]](#)

```
double f_atan_d(double t)
```

Callback derivata dell'arcotangente, restituisce $1/(1 + t^2)$.[\[file:16\]](#)[\[file:32\]](#)

```
double f_sin2plus1(double t)
```

Callback che restituisce $1/(1 + \sin^2(t))$.[\[file:16\]](#)[\[file:32\]](#)

```
double fcallb(double t, double (*f)(double))
```

Wrapper generico per valutare una callback reale f nel punto t .[\[file:16\]](#)[\[file:32\]](#)

funzioni macro algebra lineare

```
Vec linear_subst_BW(const Mat &U, const Vec &y)
```

Risolve un sistema triangolare superiore $Ux = y$ per sostituzione all'indietro.[\[file:16\]](#)[\[file:32\]](#)

Note implementative

- Implementazione didattica $O(n^2)$ senza pivoting né controlli strutturali sulla triangularità.[file:32]
- La routine assume elementi diagonali non nulli; è quindi adatta a fattorizzazioni già validate a monte.[file:32]

Vec linear_subst_FW(const Mat &L, const Vec &b)

Risolve un sistema triangolare inferiore $Lx = b$ per sostituzione in avanti.[file:16][file:32]

Note implementative

- Algoritmo classico $O(n^2)$ su matrice inferiore con diagonale non nulla.[file:32]
- Non sono previsti pivoting o controlli di degenerazione oltre alla divisione diretta sui pivot.[file:32]

void linear_jacobi_autoval_simmetrica(const Mat& A, Vec& lambda, Mat& V)

Calcola autovalori e autovettori di una matrice simmetrica reale mediante iterazioni di Jacobi con annullamento progressivo dei termini fuori diagonale.[file:16][file:32]

Note implementative

- La matrice viene copiata localmente e diagonalizzata tramite rotazioni piane, aggiornando simultaneamente la matrice degli autovettori **V**. [file:32]
- Il criterio d'arresto usa il massimo elemento fuori diagonale confrontato con una tolleranza interna **1e-12**, con limite massimo di **100 * n * n** iterazioni.[file:32]

Vec linear_LU_calcola_autovalori (const Mat& A)

Restituisce una stima degli autovalori di **A** come elementi diagonali del fattore **U** ottenuto da una fattorizzazione LU.[file:16][file:32]

Note implementative

- È una procedura euristica e non generale; il sorgente stesso la considera adatta soprattutto a matrici simmetriche e positive semidefinite in contesti sperimentali.[file:32]
- L'accuratezza dipende fortemente dall'assenza di pivoting e dalla buona condizione della matrice.[file:32]

void linear_LU_dec(const Mat &A, Mat &L, Mat &U)

Calcola una fattorizzazione LU senza pivoting della matrice quadrata **A**, producendo **L** unitaria inferiore e **U** triangolare superiore.[file:16][file:32]

Note implementative

- La procedura modifica **U** in place e costruisce **L** tramite moltiplicatori elementari di eliminazione gaussiana.[file:32]
- Se incontra un pivot nullo, solleva eccezione perché il pivoting non è implementato.[file:32]

Mat linear_LU_inversa (const Mat& A)

Calcola l'inversa di **A** risolvendo, colonna per colonna, i sistemi $Ax = e_j$ tramite la fattorizzazione LU.[file:16][file:32]

Note implementative

- Ogni colonna dell'inversa viene ottenuta con una doppia sostituzione triangolare a partire dal versore canonico corrispondente.[file:32]
- Il metodo è chiaro e adatto alla documentazione didattica, ma eredita i limiti della LU senza pivoting.[file:32]

`Vec linear_LU_risolve_colonna(const Mat& L, const Mat& U, Vec x)`

Risolve un sistema già fattorizzato **LU**, applicando prima la sostituzione in avanti e poi quella all'indietro.
[file:16][file:32]

Note implementative

- La firma accetta il termine noto per valore, ma la routine lo usa concettualmente come vettore generico del secondo membro.[file:16][file:32]
- Nel sorgente sono presenti stampe di debug dei vettori intermedi **y** e **x**. [file:32]

`Vec linear_LU_risolve_sistema(const Mat& T, const Vec& x)`

Risolutore ad alto livello che fattorizza **T** in LU e risolve il sistema lineare con termine noto **x**. [file:16][file:32]

`double linear_max_autoval_pwr_any_res(const Mat& M, int max_iter, double tol)`

Stima il massimo autovalore di $M^T M$ senza costruirlo esplicitamente, usando un metodo delle potenze con controllo di convergenza basato sulla variazione della norma del vettore iterato.[file:16][file:32]

Note implementative

- Il vettore iniziale è casuale gaussiano e viene normalizzato in norma 2.[file:32]
- Il metodo lavora con prodotti successivi $A*q$ e $A^T*(A*q)$, evitando la formazione esplicita di $M^T M$. **tol** controlla il criterio d'arresto, **max_iter** il numero massimo di iterazioni.[file:32]

`double linear_max_autoval_pwr_any(const Mat& M, int max_iter, double tol)`

Stima il massimo autovalore di $M^T M$ tramite metodo delle potenze, con quoziente di Rayleigh valutato sull'iterato normalizzato.[file:16][file:32]

Note implementative

- Anche questa variante evita la costruzione esplicita di $M^T M$, ma usa una logica di arresto sulla differenza tra stime successive dell'autovalore.[file:32]
- È utile quando interessa una stima della norma spettrale di una matrice rettangolare tramite $\|M\|_2 = \sqrt{\lambda_{\max}(M^T M)}$. [file:32]

`double linear_max_autoval_pwr_AtA(const Mat& M, int max_iter, double tol)`

Applica il metodo delle potenze a una matrice quadrata già interpretata come $A^T A$, restituendo una stima del suo autovalore dominante.[file:16][file:32]

Note implementative

- Il vettore iniziale è uniforme e normalizzato.[file:32]
- La procedura è più semplice concettualmente, ma richiede che la matrice di input sia già stata costruita esplicitamente.[file:32]

funzioni per la gestione di vettori e matrici

```
void matrix_build_cauchy(int n, double t0, double T, double z_bc, Vec &t,
Vec &avals, Vec &fvals, Mat &L, Vec &b, double (*at)(double) = at_one,
double (*ft)(double) = ft_zero, bool backw = false)
```

Costruisce il sistema lineare $Lx = b$ associato a un problema di Cauchy lineare scalare del primo ordine con coefficiente $at(t)$ e termine forzante $ft(t)$, cioè nella forma $z'(t) = a(t) * z(t) + f(t)$ discretizzato su una griglia di n nodi nell'intervallo $[t_0, T]$. [file:16][file:32]

La routine costruisce:

- il vettore dei nodi t ;
- il vettore $avals$ dei campioni del coefficiente $a(t)$;
- il vettore $fvals$ dei campioni del termine noto $f(t)$;
- la matrice bidiagonale L ;
- il termine noto b . [cite:1]

Signature

```
void matrix_build_cauchy(
    int n,
    double t0,
    double T,
    double z_bc,
    Vec &t,
    Vec &avals,
    Vec &fvals,
    Mat &L,
    Vec &b,
    double (*at)(double),
    double (*ft)(double),
    bool backw
);
```

Parametri

-

```
void matrix_build_cauchy( int n, double t0, double T, double z_bc, Vec &t, Vec &avals, Vec &fvals, Mat &L, Vec
&b, double (*at)(double), double (*ft)(double), bool backw );
```

Parametri

- ``n``: numero di nodi della discretizzazione.
- ``t0``: estremo iniziale dell'intervallo.
- ``T``: estremo finale dell'intervallo.
- ``z_bc``: valore assegnato al bordo, iniziale o finale a seconda del verso di integrazione.
- ``t``: vettore dei nodi equispaziati.
- ``avals``: campioni del coefficiente ``a(t)`` sui nodi.
- ``fvals``: campioni del termine noto ``f(t)`` sui nodi.
- ``L``: matrice del sistema lineare discreto.
- ``b``: termine noto del sistema lineare discreto.
- ``at``: puntatore a funzione per il coefficiente ``a(t)``; se ``nullptr``, viene sostituito con una funzione costante unitaria.
- ``ft``: puntatore a funzione per il termine noto ``f(t)``; se ``nullptr``, viene sostituito con una funzione costante nulla.
- ``backw``: selettore del verso di costruzione; ``false`` per schema forward, ``true`` per schema backward.[cite:1]

Convenzioni sulle callback

La routine assume callback con signature compatibile ``double(double)``. [file:16]

Sono previste anche funzioni costanti di supporto come:

```
```cpp
double at_zero(double /*t*/) { return 0.0; }
double at_one(double /*t*/) { return 1.0; }
double ft_zero(double /*t*/) { return 0.0; }
```

In particolare:

- 

```
double at_zero(double /t/) { return 0.0; } double at_one(double /t/) { return 1.0; } double ft_zero(double /t/) { return 0.0; }
```

In particolare:

- ``at_zero`` realizza il caso di integrazione pura ``(a(t)=0)``;
- ``at_one`` realizza il caso ``(a(t)=1)``;
- ``ft_zero`` realizza il caso omogeneo ``(f(t)=0)``. [cite:1]

### ### Schema forward

Nel caso ``backw == false``, la routine costruisce un sistema bidiagonale inferiore associato alla discretizzazione forward del problema con dato iniziale.

La riga interna del sistema è

$$\begin{bmatrix} z_i - \bigl(1 + h \, a(t_{i-1})\bigr) z_{i-1} = h \, f(t_{i-1}), \\ \quad i = 1, \dots, n-1 \end{bmatrix}$$

mentre la prima riga impone il dato iniziale

$$\begin{bmatrix} z_0 = z_{bc}. \end{bmatrix}$$

In questo caso la matrice è triangolare inferiore e può essere risolta con ``linear_subst_FW(...)``.[\[cite:1\]](#)

### ### Schema backward

Nel caso ``backw == true``, la routine costruisce un sistema bidiagonale superiore associato alla discretizzazione backward del problema con dato finale.

La riga interna del sistema è

$$\begin{bmatrix} \bigl(-1 - h \, a(t_i)\bigr) z_i + z_{i+1} = h \, f(t_i), \\ \quad i = 0, \dots, n-2 \end{bmatrix}$$

mentre l'ultima riga impone il dato finale

$$\begin{bmatrix} z_{n-1} = z_{bc}. \end{bmatrix}$$

In questo caso la matrice è triangolare superiore e può essere risolta con ``linear_subst_BW(...)``.[\[cite:1\]](#)

### ### Casi particolari

La stessa routine copre automaticamente diversi casi notevoli:

- **\*\*integrazione pura\*\***:  $(z'(t)=f(t))$ , ottenuta con  $(a(t)=0)$ ;
  - **\*\*equazione omogenea\*\***:  $(z'(t)=a(t)z(t))$ , ottenuta con  $(f(t)=0)$ ;
  - **\*\*caso separabile testato\*\***:  $(z'(t)=z(t)+f(t))$ , ottenuto con  $(a(t)=1)$ .
- [\[cite:1\]](#)

In questo modo la distinzione tra i diversi tipi di ODE non è affidata a funzioni pubbliche differenti, ma emerge direttamente dalla formula discreta e dai campioni delle callback ``at`` e ``ft``.[\[cite:1\]](#)

### ### Note implementative

La routine inizializza internamente i vettori ``t``, ``avals``, ``fvals``, la matrice ``L`` e il vettore ``b``, così da evitare dipendenze da contenuti residui già presenti nei contenitori passati per riferimento.[\[file:32\]](#)



Il passo di discretizzazione è

```
\[
h = \frac{T - t_0}{n - 1}.
\]
```

I nodi sono quindi costruiti come

```
\[
t_i = t_0 + i \cdot h,
\quad i = 0, \dots, n-1.
\]
```

Questa scelta mantiene una corrispondenza diretta tra nodo, campionamento dei coefficienti e riga discreta del sistema lineare.[cite:1]

```
`void matrix_build_cauchy_int(int n, double t0, double T, double x0, Vec &t,
Vec &fvals, Mat &L, Vec &b, double (*ft)(double), bool backw)`
Versione semplificata di `matrix_build_cauchy` con coefficiente `at(t) ≡ 0`
(equazione di integrazione pura `z' = f(t)`). Campiona `ft` internamente e
costruisce il sistema bidiagonale corrispondente. Deprecata dopo il collaudo di
`matrix_build_cauchy`. [file:16][file:32]
```

**Note implementative**

- Caso limite con solo termine forzante; equivalente a un'integrazione numerica di `f(t)` con schema alle differenze finite.[file:32]

```
`void matrix_build_cauchy_sep(int n, double t0, double T, double x0, Vec &t,
Vec &fvals, Mat &L, Vec &b, double (*ft)(double), bool backw)`
Variante per ODE a variabili separabili `(ft ≡ 0)` : costruisce il sistema omogeneo
`z' - a(t)z = 0`.
Utile come banco di test per confronto con la soluzione esatta analitica `z(t) =
z_bc * exp(∫a(s)ds)`. Deprecata dopo il collaudo di `matrix_build_cauchy`.
[file:16][file:32]
```

```
`Mat matrix_build_derivata1(int n, double a, double b)`
Costruisce una matrice delle differenze finite in avanti di dimensione `(n-1) × n` per approssimare la derivata prima su una griglia uniforme.[file:16][file:32]
```

**Note implementative**

- Gli elementi non nulli sono  $(-1/h)$  sulla diagonale e  $(1/h)$  sulla sopradiagonale immediata.[file:32]
- Il passo `h` è ottenuto da `h\_ticks(a, b, n - 1)`. [file:32]

```
`Mat matrix_build_gausskernel(const int N, const double sigma, const double h,
bool norm_flag, Vec& Indicatori)`
Costruisce una matrice kernel gaussiana campionata su una griglia uniforme, con
possibilità di normalizzazione per righe e restituzione di indicatori numerici.
[file:16][file:32]
```

**Note implementative**

- L'elemento  $(K_{ij})$  è ottenuto da una gaussiana centrata sulla differenza  $x_i$

```

- xj`.[file:32]
- Se `norm_flag` è attivo, le righe vengono normalizzate. Il vettore `Indicatori`
viene riempito con quantità diagnostiche legate alla norma della matrice, della
sua inversa e al numero di condizionamento approssimato.[file:32]

`Mat matrix_build_gram(const Vec& x, const int K)`
Costruisce una matrice di Gram `K x K` campionando una famiglia di funzioni sui
nodi `x` e usando prodotti scalari tra campioni.[file:16][file:32]

Note implementative
- Nel sorgente i campioni sono ottenuti con `vector_campiona_f_k(..., f_cos)` e la
matrice viene riempita in modo simmetrico.[file:32]
- La routine è utile come banco di prova per ortogonalità e dipendenza lineare di
basi discrete.[file:32]

`Mat matrix_build_Id(int n)`
Costruisce la matrice identità `n x n`.[file:16][file:32]

`Mat matrix_build_triang(int n)`
Costruisce una matrice triangolare inferiore piena di uni.[file:16][file:32]

`Mat matrix_build_triang_inv(int n)`
Costruisce una matrice triangolare inferiore che rappresenta l'inversa della
matrice triangolare di uni del caso precedente, con `1` in diagonale e `-1` sulla
sottodiagonale immediata.[file:16][file:32]

`Mat matrix_build_zero(int righe, int colonne)`
Costruisce una matrice nulla delle dimensioni richieste.[file:16][file:32]

`Vec matrix_calcola_deriv_byiter(const Vec &u, const double a, const double
b)`
Approssima la derivata del vettore `u` campionato sull'intervallo `[a, b]` tramite
differenze finite iterative, senza costruire esplicitamente la matrice delle
differenze.[file:16][file:32]

`Vec matrix_calcola_deriv_bymatr(const Mat &D, const Vec &u)`
Calcola il prodotto matrice-vettore `D * u` dove `D` è la matrice delle differenze
finite pre-costruita (tipicamente da `matrix_build_derivata1`). Approccio
matriciale equivalente a `matrix_calcola_deriv_byiter`.[file:16][file:32]

Note implementative
- Le due varianti (`byiter` e `bymatr`) producono risultati identici sulla stessa
griglia e sono mantenute entrambe come confronto didattico tra approccio iterativo
e approccio matriciale.[file:32]

`double matrix_calcola_errore_Fr(const Mat& A, const Mat& B)`
Calcola la norma di Frobenius della differenza `A - B`.[file:16][file:32]

`void matrix_calcola_media(const Mat& A, Vec& avg_col, Vec& avg_row)`
Calcola le medie per colonna e per riga della matrice `A`.[file:16][file:32]

`double matrix_calcola_norma(int norma, const Mat& A)`
Calcola varie norme matriciali e alcune stime della norma 2, in funzione del
codice intero `norma`.[file:16][file:32]

```

**\*\*Note implementative\*\***

- `norma = 1` restituisce la norma 1, `norma = 0` la norma infinito, `norma = -1` la norma di Frobenius.[file:32]
- `norma = 2`, `12` e `22` attivano diverse strategie per stimare la norma 2, basate rispettivamente su LU sperimentale o su metodi delle potenze con o senza costruzione esplicita di  $(A^T A)$ . [file:32]
- La presenza di più codifiche rende utile documentare esplicitamente il significato operativo del parametro nelle note a margine del documento finale. [file:32]

### `Mat matrix\_centra\_su\_media(const Mat& A, const Vec& avg\_vec, bool by\_col = true)`

Centra la matrice sottraendo a ciascun elemento la media di colonna o di riga, a seconda del flag `by\_col`. [file:16][file:32]

### `Mat matrix\_differenza\_dump(const Mat& A, const Mat& B)`

Restituisce la differenza `A - B` se le dimensioni sono compatibili; in caso contrario termina il programma con messaggio diagnostico. [file:16][file:32]

### `void matrix\_dump(const Mat &A, const std::string &nome)`

Stampa in console una matrice formattata, con colorazione degli elementi in funzione del loro segno e ordine di grandezza. [file:16][file:32]

### `Mat matrix\_estende\_ridotta(const Mat& A, int n, bool bycol)`

Riduce o mantiene una matrice alle dimensioni richieste, tagliando righe o colonne a seconda del flag `bycol`. [file:16][file:32]

### `Mat matrix\_normalize\_byrow(Mat& K)`

Restituisce una copia della matrice in cui ogni riga è normalizzata rispetto alla propria somma degli elementi. [file:16][file:32]

### `void matrix\_ordina\_diagonale(Vec& lambda, Mat& V, double zero\_tol = 0.0, SortOrder order = SortOrder::Desc)`

Ordina il vettore `lambda` e trascina coerentemente le colonne della matrice `V`, con possibile azzeramento preliminare dei valori inferiori alla soglia `zero\_tol`. [file:16][file:32]

**\*\*Note implementative\*\***

- La routine è centrale nella pipeline SVD/autovalori, perché consente di riordinare autovalori e autovettori mantenendo l'allineamento colonnare. [file:32]
- Il parametro `order` distingue ordinamento crescente e decrescente. [file:16][file:32]

### `void matrix\_ortogonalizza\_GSmod(Mat& Q, int j0 = 0)`

Ortonormalizza le colonne di `Q` con Gram-Schmidt modificato, a partire dalla colonna `j0`. [file:16][file:32]

**\*\*Note implementative\*\***

- Serve sia come routine autonoma sia come supporto al completamento di basi ridotte in contesti SVD. [file:32]
- Le colonne quasi nulle vengono lasciate a zero anziché generare errore. [file:32]

### `Mat matrix\_prodotto\_coeff(const Mat& A, const double coeff)`

Restituisce il prodotto scalare ``coeff * A``. [file:16][file:32]

### ``Mat matrix_prodotto_AtA(const Mat& A, bool A_right = true)``  
 Costruisce ``A^T A`` se ``A_right`` è ``true``, oppure ``A A^T`` se ``A_right`` è ``false``.  
 [file:16][file:32]

### ``Mat matrix_prodotto_matrix(const Mat& A, const Mat& B)``  
 Calcola il prodotto matrice-matrice ``A * B``, con controllo di compatibilità dimensionale e lancio di eccezione in caso di mismatch. [file:16][file:32]

### ``Vec matrix_prodotto_vector(const Mat& A, const Vec& v)``  
 Calcola il prodotto matrice-vettore ``A * v``, con controllo di compatibilità dimensionale. [file:16][file:32]

### ``void matrix_test_ortogonale(const Mat& A, string s)``  
 Esegue un test di ortogonalità stampando il prodotto ``A^T A`` associato alla matrice ``A``. [file:16][file:32]

### ``Mat matrix_trasposta(const Mat& A)``  
 Restituisce la trasposta della matrice ``A``. [file:16][file:32]

### ``Vec vector_add_noise(Vec& v, double e)``  
 Aggiunge rumore casuale al vettore ``v``, con ampiezza proporzionale alla sua norma e al parametro ``e`` espresso in millesimi. [file:16][file:32]

### ``Vec vector_build_versore_canonico(int j, int N)``  
 Costruisce il versore canonico ``e_j`` di dimensione ``N``. [file:16][file:32]

### ``double vector_calcola_norma(int norma, const Vec& V)``  
 Calcola la norma 2 o la norma infinito del vettore, a seconda del codice ``norma``.  
 [file:16][file:32]

### ``Vec vector_campiona_ft(const Vec &x, double (*ft)(double))``  
 Campiona una funzione reale ``ft`` sui nodi contenuti in ``x``. [file:16][file:32]

### ``Vec vector_campiona_f_k(int k, const Vec &x, double (*ft)(double))``  
 Campiona una funzione base parametrica sui nodi ``x``, usando ``k`` come fattore di frequenza o parametro discreto di famiglia. [file:16][file:32]

**\*\*Note implementative\*\***

- Nel sorgente la callback viene richiamata come ``fcallb(k * x[i], ft)``, quindi ``k`` agisce come acceleratore della variabile di input. [file:32]

### ``void vector_dump(Vec x, int colspan, int totnum, const std::string s)``  
 Stampa in console il contenuto del vettore con impaginazione a colonne, utile per debug e ispezione manuale. [file:16][file:32]

### ``Vec vector_householder_bycol(const Vec& v)``  
 Costruisce il vettore di Householder associato a una colonna, in modo che la riflessione annulli tutti gli elementi sotto il primo. [file:16][file:32]

**\*\*Note implementative\*\***

- Se il vettore ha norma nulla, restituisce il primo versore canonico come trasformazione neutra. [file:32]

- La convenzione di segno è scelta per aumentare la stabilità numerica nella costruzione di  $\vec{u} = \vec{v} + \text{sign}(v_0)|\vec{v}|e_1$ . [file:32]

### ``Vec` vector_householder_byrow(const Vec& v)``

Versione per righe del costruttore di Householder; nel codice delega direttamente alla versione per colonne. [file:16][file:32]

### ``double` vector_prodotto_scalare(const Vec &u, const Vec &v)``

Calcola il prodotto scalare euclideo tra due vettori della stessa dimensione. [file:16][file:32]

### ``Vec` vector_reverse(const Vec& v)``

Restituisce una copia del vettore con ordine degli elementi invertito. [file:16][file:32]

### ``Vec` vector_segnalet_finestra(int N, double a, double b, double t)``

Genera un segnale finestrato di lunghezza  $N$ , con valore  $t$  nell'intervallo relativo  $[a, b]$  e zero altrove. [file:16][file:32]

### ``Vec` vector_shift(const Vec &v, const double shift)``

Restituisce una copia del vettore traslata di una quantità costante  $\text{shift}$ . [file:16][file:32]

### ``Mat` vector_to_matrix(const Vec& v, bool transp)``

Converte un vettore in matrice colonna oppure in matrice riga, a seconda del flag  $\text{transp}$ . [file:16][file:32]

### ``Mat` vector_to_matrix_diag(const Vec& s, int m, int n)``

Costruisce una matrice diagonale  $m \times n$  usando gli elementi del vettore  $s$  sulla diagonale principale fino alla minima dimensione disponibile. [file:16][file:32]

## funzioni specifiche di trasformazione matrici

### ``void` trmatrix_bidiagonalizza(const Mat& A, Mat& U0, Mat& B, Mat& V0, bool sup_diag = false, bool dump_flag = false)``

Wrapper generale per la bidiagonalizzazione di una matrice rettangolare, con restituzione di fattori ortogonali  $U_0$ ,  $V_0$  e matrice bidiagonale  $B$  tali che  $A = U_0 B V_0^T$ . [file:16][file:32]

**\*\*Note implementative\*\***

- La routine distingue tra caso "wide" e caso "tall"; nel secondo usa la trasposta e riconduce il problema alle funzioni per matrici wide. [file:32]
- Il parametro  $\text{sup\_diag}$  seleziona bidiagonale superiore o inferiore, mentre  $\text{dump\_flag}$  abilita stampe diagnostiche intermedie. [file:16][file:32]

### ``void` trmatrix_bidiag_wide_to_lower(const Mat& X, Mat& U0, Mat& B, Mat& V0, bool dump_flag)``

Riduce una matrice wide a forma bidiagonale inferiore tramite una sequenza di riflessioni di Householder applicate prima a destra e poi a sinistra. [file:16][file:32]

**\*\*Note implementative\*\***

- Gli aggiornamenti di  $U_0$  e  $V_0$  sono espliciti e mantengono memoria del prodotto cumulativo delle trasformazioni ortogonali. [file:32]

- La funzione è pensata anche come strumento didattico, perché ``dump_flag`` consente di seguire passo passo struttura della matrice e ortogonalità dei fattori.[file:32]

```
`void trmatrix_bidiag_wide_to_upper(const Mat& X, Mat& U0, Mat& B, Mat& V0,
bool dump_flag)`
```

Riduce una matrice wide a forma bidiagonale superiore applicando Householder su sotto-colonne e sotto-righe in successione tipo Golub-Kahan.[file:16][file:32]

**\*\*Note implementative\*\***

- La procedura usa la formula efficiente  $(H A = A - 2 w (w^T A))$  e aggiorna i blocchi attivi della matrice senza costruire esplicitamente l'intera riflessione.[file:32]

- Anche in questo caso ``dump_flag`` abilita il tracciamento dei passaggi e dei test di ortogonalità di ``U0`` e ``V0``.[file:32]

```
`void trmatrix_SVDQR(const Mat& B, Mat& Ub, Mat& Vb, Vec& sigma)`
```

Wrapper ad alto livello per la SVD della matrice ``B``, che richiama la versione ridotta e completa quando necessario la base destra.[file:16][file:32]

**\*\*Note implementative\*\***

- La funzione mantiene l'interfaccia compatibile con la routine SVD precedente, riordinando i parametri di uscita e completando ``Vb`` a base integrale quando la decomposizione nasce in forma ridotta.[file:32]

```
`void trmatrix_SVDQR_ridotta(const Mat& B, Mat& Ub, Vec& sigma, Mat& Vb_red,
double ev_tol = 1e-12)`
```

Calcola una SVD ridotta di ``B`` mediante diagonalizzazione di  $(B^T B)$ , ordinamento degli autovalori e ricostruzione dei vettori singolari sinistri da  $(u_i = (1/\sigma_i) B v_i)$ .[file:16][file:32]

**\*\*Note implementative\*\***

- Gli autovalori di  $(B^T B)$  sono calcolati da ``linear_jacobi_autoval_simmetrica``, poi ordinati con ``matrix_ordina_diagonale``.[file:32]

- Il parametro ``ev_tol`` serve a schiacciare a zero autovalori negativi di origine numerica o quasi nulli, evitando artefatti nel calcolo di ``sqrt(lambda)``.[file:16][file:32]

- La procedura produce ``sigma`` e ``Vb_red`` in forma ridotta; eventuali completamenti ortogonali vengono demandati al livello superiore.[file:32]

```
`void trmatrix_test_sv_autoval(Vec lambda, const Vec& sigma)`
```

Confronta autovalori e valori singolari stampando gli scarti tra ``lambda`` e ``sigma^2``, oltre al confronto tra ``sigma`` e ``sqrt(lambda)``.[file:16][file:32]

**## Funzioni per matplotlib++**

```
`void matplotlib_legend_align(legend_handle lg, int pos_enum, float xscale, float
yscale)`
```

Allinea e riposiziona la legenda Matplotlib secondo alcune configurazioni discrete codificate da ``pos_enum``.[file:16][file:32]

**\*\*Note implementative\*\***

- Le opzioni implementate includono posizionamento centrato superiore esterno, in

basso a destra, al centro a sinistra e in alto a sinistra.[file:32]

```
`figure_handle matplotlib_table_init(const bool ahold, const std::string &nome,
const std::string &titolo, const int xlab, const int ylab)`
Inizializza una figura Matplotlib con layout tiled, dimensioni fissate, nome
finestra e titolo configurato.[file:16][file:32]
```

```
gestione menu principale
```

```
`struct MenuItem`
```

Struttura dati che rappresenta un elemento di menu con chiave numerica, etichetta, azione associata e flag di abilitazione.[file:16]

```
`struct MenuConfig`
```

Struttura dati che rappresenta una configurazione di menu completa, con titolo e lista di `MenuItem`. [file:16]

```
`MenuConfig load_menu_config(const std::string& filename)`
```

Carica una configurazione di menu da file JSON e costruisce la struttura `MenuConfig` corrispondente.[file:16][file:32]

**\*\*Note implementative\*\***

- La routine usa `nlohmann::json`, verifica la presenza dell'array `items` e lancia eccezioni in caso di file non accessibile o struttura non valida.[file:32]
- Nel sorgente sono presenti messaggi di debug su working directory e dimensione del file letto.[file:32]

```
`const MenuItem* find_menu_item(const MenuConfig& menu, int key)`
```

Restituisce un puntatore all'elemento di menu con chiave `key`, oppure `nullptr` se non trovato.[file:16][file:32]

```
`void wait_return_to_menu(bool bypass_waitakey)`
```

Attende il tasto INVIO prima del ritorno al menu, salvo disattivazione esplicita tramite `bypass\_waitakey`. [file:16][file:32]